Strings and How They Relate to Lists

CMSC 104 Section 2
October 26, 2026

Administrative Notes

Classwork 6 is due Wednesday

What you should take from this lecture

With strings you can:

- Treat characters individually
- Manipulate strings in some ways
- Use built-in methods
  - len, upper, lower, …
- Reference individual characters in a string via indexing
  - Emphasize that you always start counting at 0
  - You can use negative index values to start counting from the end instead of the beginning
- Take substrings via string slicing
- Concatenating strings together (you've been doing this already)
- Repeating strings via "multiplication"

But you CANNOT:

Modify a string in place by changing specific characters

This is because strings are ***immutable*** - unlike lists.

Strings to date

They’re a single variable

- A sequence of 0 or more characters, treated as a whole
  - A “character” is whatever can be represented by bits in your computer/Python instance
    - ASCII, UTF-8, UTF-16, etc.
    - We’re not diving into that, but you may run across it later in your career

We can have string literals

We can assign values to variables

We can compare string values

```
if college == “UMBC”:
```

But it turns out we can do a lot more with strings

Every character in a string can be accessed individually

We can determine if a specific character appears in a string

We can calculate the length of a string

We can convert all the characters in a string to UPPERCASE or lowercase or…

- This will not impact characters that aren't letters

See the file called "strings.py" in the code samples repo on the course GitHub repo

Predefined methods for manipulating strings in Python

Method	What it does
.upper()	Converts each letter in the string to uppercase
.lower()	Converts each letter in the string to lowercase
.title()	Converts the entire string to Title Case
.strip()	Removes leading and/or trailing blank spaces
.split()	Splits a string into a list of strings, and removes all instances of the character(s) in the method
.replace()	Replaces all instances of a substring with a new substring

What can’t you do?

You cannot directly change the individual characters in a string

```
college = “UMBD”
college[3] = “C”
```

This will fail. You can’t do this directly. Why? Because strings are ***immutable variables***. They can’t be changed; they can only be reassigned.

So what can you do?

- You can break up the string into multiple variables; then reassign the variables you want, then compose them back into a string
- You can use the “replace” method, which does exactly that, except it’s been pre-written for you by somebody else